Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

# ОТЧЁТ
по лабораторной работе №6
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Интеграция с NoSQL базой данных MongoDB

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

**СОДЕРЖАНИЕ**

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 4
3.1 Подключение к базе данных 4
3.2 Схема и модель 4
3.3 CRUD через Mongoose 5
3.4 Работа с вложенными структурами 6
3.5 Запуск и проверка 7
3.6 Демонстрация работы 8
ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 24
ЗАКЛЮЧЕНИЕ 25
ПРИЛОЖЕНИЕ А Листинг файла db.js 26
ПРИЛОЖЕНИЕ Б Листинг файла server.js 27
ПРИЛОЖЕНИЕ В Листинг файла diagram.js 28
ПРИЛОЖЕНИЕ Г Листинг файла diagrams.controller.js 30
ПРИЛОЖЕНИЕ Д Листинг файла diagrams.routes.js 33
ПРИЛОЖЕНИЕ Е Листинг файла seed.js 34
ПРИЛОЖЕНИЕ Ж Листинг файла limits.json 35

## 1 ЦЕЛЬ РАБОТЫ

Освоить работу с документной базой данных MongoDB: подключение, описание схем и моделей через ODM Mongoose, CRUD-операции в маршрутах Express и вложенные структуры документа. Работа выполнена для проекта DSLKit, веб-редактора диаграмм как кода, тематика которого соответствует направлению SaaS. Это альтернативная реализация хранения коллекции `diagrams` из лабораторной работы 2: тот же REST API, но вместо таблицы реляционной базы используется коллекция MongoDB. Код лежит в ветке репозитория `26`.

## 2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

**Документная модель.** MongoDB хранит данные в коллекциях, а коллекция состоит из документов в формате BSON, двоичном расширении JSON. У каждого документа есть уникальное поле `_id`, по умолчанию это `ObjectId` длиной 12 байт: время создания, случайное значение и счётчик. Документ может содержать вложенные объекты и массивы, поэтому данные, которые в реляционной базе разнесены по нескольким таблицам, хранятся вместе и читаются одним запросом.

**Mongoose.** ODM Mongoose добавляет к MongoDB схему и модель. Схема описывает поля, их типы, обязательность, допустимые значения и индексы, а модель даёт методы `find`, `findById`, `create`, `findByIdAndUpdate` и `findByIdAndDelete`. Схема проверяется в приложении, сама MongoDB структуру документов не навязывает.

Таблица 1 – Сравнение реляционной и документной баз данных

| **Признак** | **Реляционная (PostgreSQL)** | **Документная (MongoDB)** |
|---|---|---|
| Единица хранения | Строка таблицы | Документ BSON в коллекции |
| Схема | Жёсткая, меняется миграциями | Гибкая, правила задаёт код приложения |
| Связанные данные | Отдельные таблицы и соединения `JOIN` | Вложенные документы и массивы или ссылки и `populate` |
| Идентификатор | Целое число или UUID | `ObjectId` |
| Транзакции и связи | Внешние ключи, строгие транзакции | Атомарность в пределах одного документа |
| Масштабирование | Обычно вертикальное | Горизонтальное через шардирование |

Недостаток документной модели в дублировании данных и в слабых связях между коллекциями: каскадного удаления и внешних ключей нет, их

приходится обеспечивать кодом. Преимущество в скорости чтения целого объекта и в простоте изменения структуры.

# 3 ХОД РАБОТЫ

## 3.1 Подключение к базе данных

Адрес базы берётся из переменной окружения `MONGODB_URI`. Её читает пакет `dotenv` из файла `.env`, а образец `.env.example` лежит в репозитории, поэтому пароль к кластеру не попадает в систему контроля версий. Один и тот же код работает с локальной установкой MongoDB Community и с кластером MongoDB Atlas, меняется только строка подключения. Функция `connect` в файле `src/db.js` вызывает `mongoose.connect`, а `server.js` запускает сервер только после успешного подключения, иначе печатает причину и завершает процесс с кодом 1. Подключение к Atlas в этой работе не проверялось, запуск проверен на локальном `mongod` версии 8.2, а тесты работают на временной базе.

## 3.2 Схема и модель

Схема `Diagram` описана в `src/models/diagram.js`. Основные поля те же, что в лабораторных работах 1 и 2: `title`, `notation`, `sourceCode`, `priority` и `owner`. Допустимые значения `notation` заданы через `enum`, а `priority` это целое число от 0 до 2 со значением по умолчанию 1. Обязательные строки не принимают пустые и пробельные значения, длины строк, число тегов и число комментариев ограничены. Все пределы лежат в отдельном файле `limits.json`, который читают и схема, и проверка запросов, поэтому изменить предел можно без правки кода. Для нужд документной модели добавлены два вложенных поля: массив строк `tags` и массив вложенных документов `comments` с полями `author`, `text` и `createdAt`. Комментарий хранится внутри диаграммы, отдельной коллекции и соединений для него нет. Mongoose сам добавляет `_id` диаграммы и каждого комментария, а параметр `timestamps` ведёт `createdAt` и `updatedAt`. Для ускорения поиска объявлены индексы по `owner` и `tags`, а для ответов настроено преобразование в JSON: поле `_id` заменено на `id`, служебное `__v` скрыто.

Таблица 2 – Сопоставление таблиц реляционной версии и документа MongoDB

| **Реляционная версия (ЛР 2)** | **Документная версия (ЛР 6)** |
|---|---|
| Таблица `Diagrams`, строка на диаграмму | Коллекция `diagrams`, документ на диаграмму |
| Теги пришлось бы вынести в таблицы `Tags` и связь | Массив `tags` внутри документа |
| Комментарии в таблице `Comments` с внешним ключом | Массив `comments` внутри документа |
| Чтение диаграммы с комментариями через `JOIN` | Один запрос `findById` |
| Удаление комментариев каскадом по внешнему ключу | Удаляются вместе с документом |

### 3.3 CRUD через Mongoose

Контроллеры получают модель параметром и вызывают методы Mongoose, как показано в таблице 3. Метод `findByIdAndUpdate` вызывается с параметрами `new: true` и `runValidators: true`: клиент получает обновлённый документ, а проверки схемы срабатывают и при обновлении. Идентификатор в адресе проверяется до обращения к базе: допустима строка из 24 шестнадцатеричных символов, иначе ответ 400. Неизвестный, но корректный идентификатор даёт 404. Ошибки `ValidationError` и `CastError` от Mongoose обработчик ошибок переводит в ответ 400, а любой другой сбой в 500 без внутренних подробностей. Защита от инъекций операторов построена в три слоя. Проверка запроса принимает в полях только строки нужной длины, поэтому тело вида `{ "$ne": null }` отвергается ответом 400, а параметры списка допускают только `owner` и `tag` со строковым значением. Настройки `sanitizeFilter` и `strictQuery` не дают оператору из значения попасть в запрос, а лишние поля тела отбрасываются схемой. Размер тела запроса ограничен значением 100 КБ. Формат ошибок, ответ 405 с заголовком `Allow` и ограничение размера тела остались как в предыдущих работах.

Таблица 3 – Соответствие маршрутов и методов Mongoose

| **Маршрут** | **Вызов Mongoose** | **Ответ** |
|---|---|---|
| `GET /diagrams` | `Diagram.find(filter).sort({ _id: 1 })` | 200 |
| `GET /diagrams/:id` | `Diagram.findById(id)` | 200 или 404 |
| `POST /diagrams` | `Diagram.create(fields)` | 201 |

| Маршрут | Вызов Mongoose | Ответ |
|---|---|---|
| `PUT /diagrams/:id` | `Diagram.findByIdAndUpdate(id, fields, { new: true })` | 200 или 404 |
| `DELETE /diagrams/:id` | `Diagram.findByIdAndDelete(id)` | 204 или 404 |

Список принимает параметры `tag` и `owner`. Фильтр `{ tags: 'shop' }` в MongoDB означает «массив тегов содержит значение», отдельного оператора для этого не нужно.

### 3.4 Работа с вложенными структурами

Три дополнительных маршрута работают с массивом комментариев и меняют его отдельными атомарными операторами MongoDB, не загружая документ в приложение.

Таблица 4 – Маршруты для массива комментариев

| Маршрут | Оператор MongoDB | Ответ |
|---|---|---|
| `POST /diagrams/:id/comments` | `$push: { comments: comment }` | 201 или 404 |
| `PUT /diagrams/:id/comments/:commentId` | `$set: { 'comments.$.text': text }` с условием `comments._id` | 200 или 404 |
| `DELETE /diagrams/:id/comments/:commentId` | `$pull: { comments: { _id: commentId } }` | 200 или 404 |

Изменение одного элемента массива показывает отличие от реляционной модели: позиционный оператор `$` подставляет индекс найденного элемента, поэтому обновляется только текст нужного комментария, а остальные элементы массива и поля диаграммы не затрагиваются. Условие поиска содержит и идентификатор диаграммы, и идентификатор комментария, поэтому комментарий другой диаграммы по этому адресу недоступен. Метод `PUT /diagrams/:id` меняет поля диаграммы и теги, но не трогает комментарии.

### 3.5 Запуск и проверка

Порядок запуска: скопировать `.env.example` в `.env` и вписать `MONGODB_URI`, выполнить `npm run seed`, затем `npm run dev`. Сид удаляет демонстрационные документы по названию и вставляет три диаграммы, поэтому его можно запускать повторно. Проверка на локальном `mongod` показала, что список возвращает диаграммы с вложенными комментариями и тегами, а запрос с параметром `tag=shop` возвращает две диаграммы из трёх.

К маршрутам добавлены частичное изменение `PATCH /diagrams/:id`, сортировка `?sort=title` или `?sort=-priority`, счётчик записей (маршрут `/diagrams/count` и заголовок `X-Total-Count` в списке) и поиск `?q=` по текстовому индексу заголовка и исходного кода. Всего 144 автоматических теста. Девятнадцать проверяют основные маршруты: корректные и некорректные идентификаторы, создание, замену с сохранением комментариев, удаление, ошибки 400, 404, 405 и 413, ответ 500 без утечки текста ошибки. Одиннадцать проверяют вложенные структуры: фильтр по тегу и владельцу, добавление, изменение и удаление комментария, недоступность чужого комментария, проверку схемы на уровне модели, индексы и перевод ошибки Mongoose в ответ 400. Десять проверяют `PATCH`, сортировку, счётчик и текстовый поиск. Двадцать восемь проверяют неправильные действия пользователя: пустое, оборванное и не-JSON тело, неверные типы, пробельные значения, значения за пределами, поддельные поля, внедрение операторов, неверные идентификаторы, повторные и параллельные запросы, остановку базы. Ещё одиннадцать проверяют защиту данных: операторы вроде `$ne` в теле и в параметрах запроса, пробельные значения, пределы из `limits.json` на границе и за ней, целочисленный приоритет, ограничение размера тела и наличие времени создания у каждой записи. Ещё тридцать девять тестов проверяют дополнительные сценарии неправильного обращения, двенадцать проверяют заголовки безопасности, журнал запросов, агрегацию и поиск по вложенному полю, девять проверяют устойчивость (нагрузку, одновременные запросы, обрыв соединений, восстановление после остановки базы, кириллицу и эмодзи), три проверяют сид-данные и два проверяют маршрут `/health`. Тесты запускаются на временном экземпляре MongoDB через `mongodb-memory-server`, поэтому рабочая база не затрагивается. Те же команды `npm test` проверяют сборку и оформление этого отчёта. Скриншоты кластера Atlas, запросов в Postman и данных в MongoDB Compass в этот вариант отчёта не включены.

### 3.6 Демонстрация работы

Все снимки получены скриптом `shots/shots.mjs`: он поднимает временную MongoDB и сервер, выполняет настоящие запросы и сохраняет результат. Страницы «запрос и ответ» нарисованы этим скриптом по записанным данным обмена, это не окно Postman. Снимки 21–25 сделаны в браузере Chrome, который сам открывает адрес работающего сервера.

Список из трёх диаграмм с тегами и вложенными комментариями показан на рисунке 1.

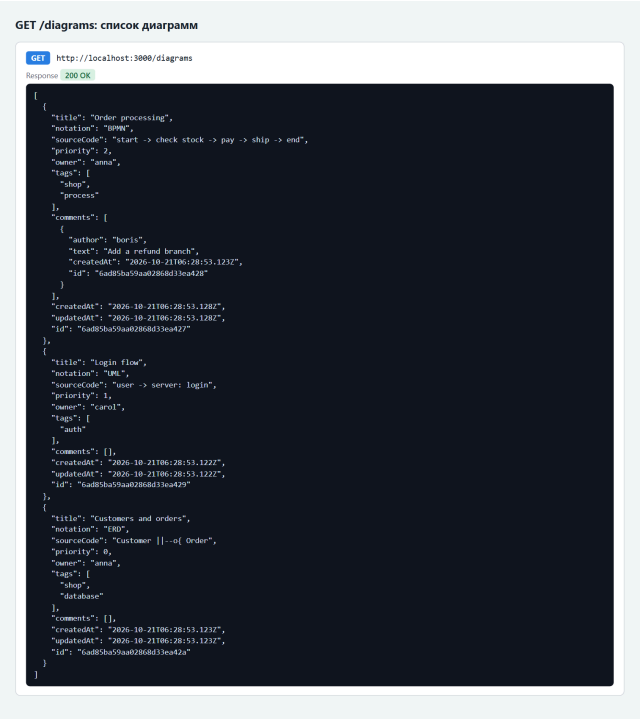



Рисунок 1 – Запрос GET /diagrams и ответ 200, записанные скриптом

Запрос по идентификатору ObjectId возвращает одну диаграмму, как на рисунке 2.

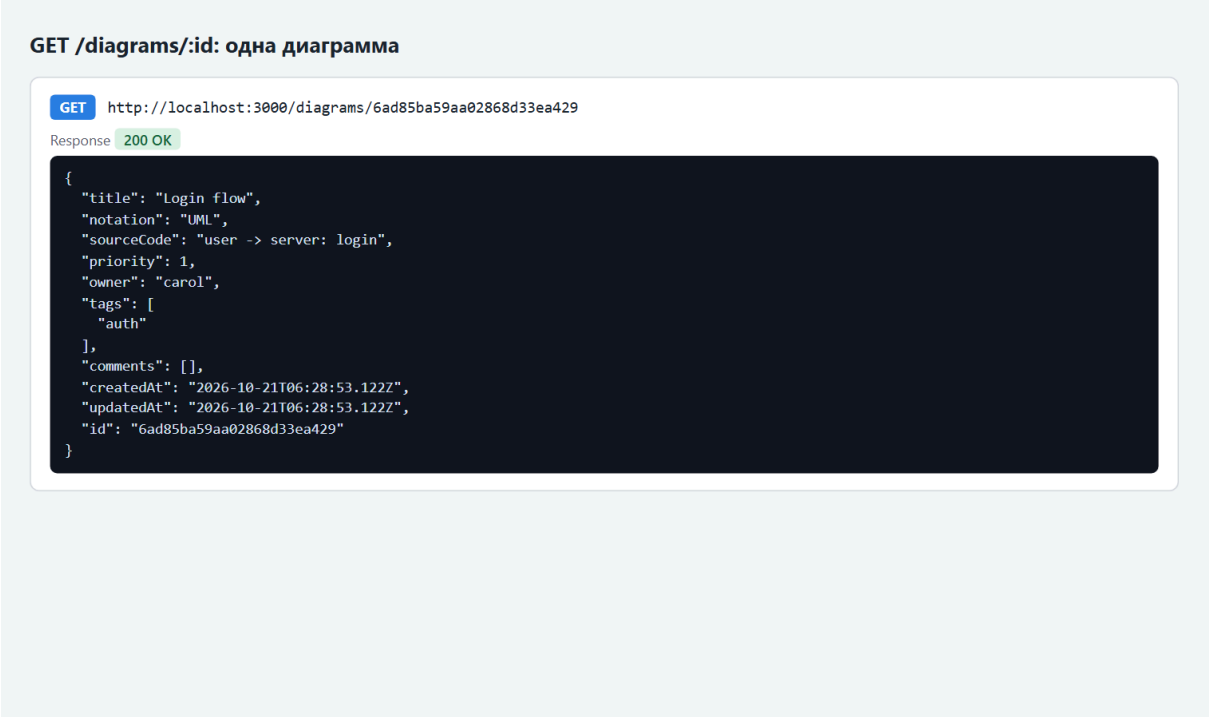



Рисунок 2 – Запрос GET /diagrams/:id и ответ 200, записанные скриптом

Создание диаграммы возвращает код 201 и заголовок Location, что видно на рисунке 3.

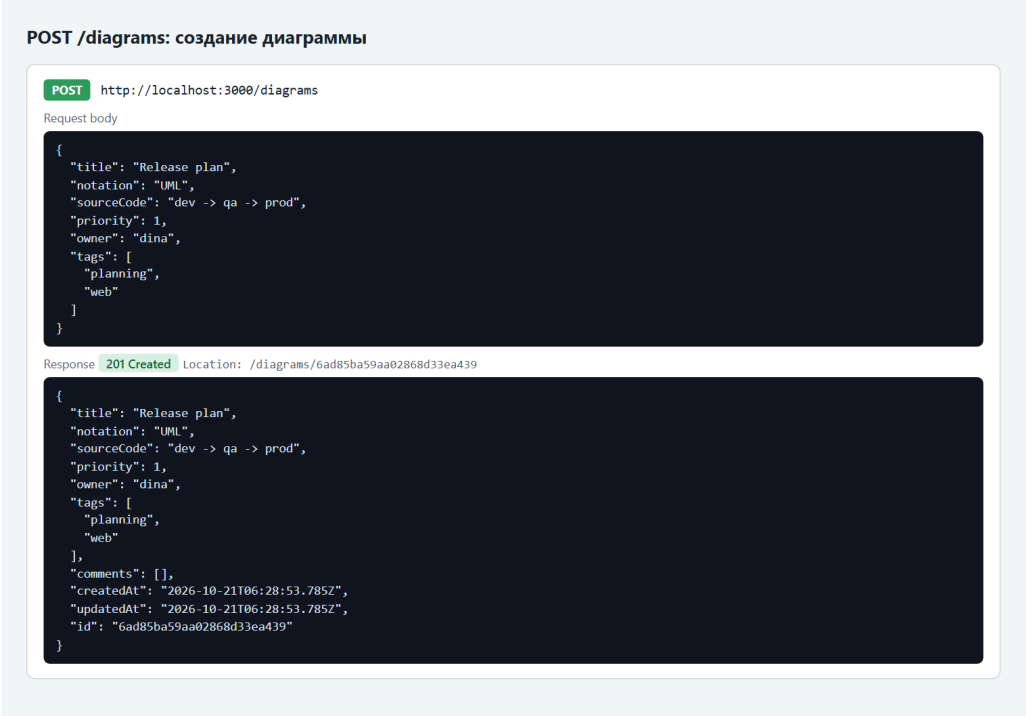



Рисунок 3 – Запрос POST /diagrams и ответ 201, записанные скриптом

Метод PUT заменяет поля, а комментарий диаграммы сохраняется, как на рисунке 4.

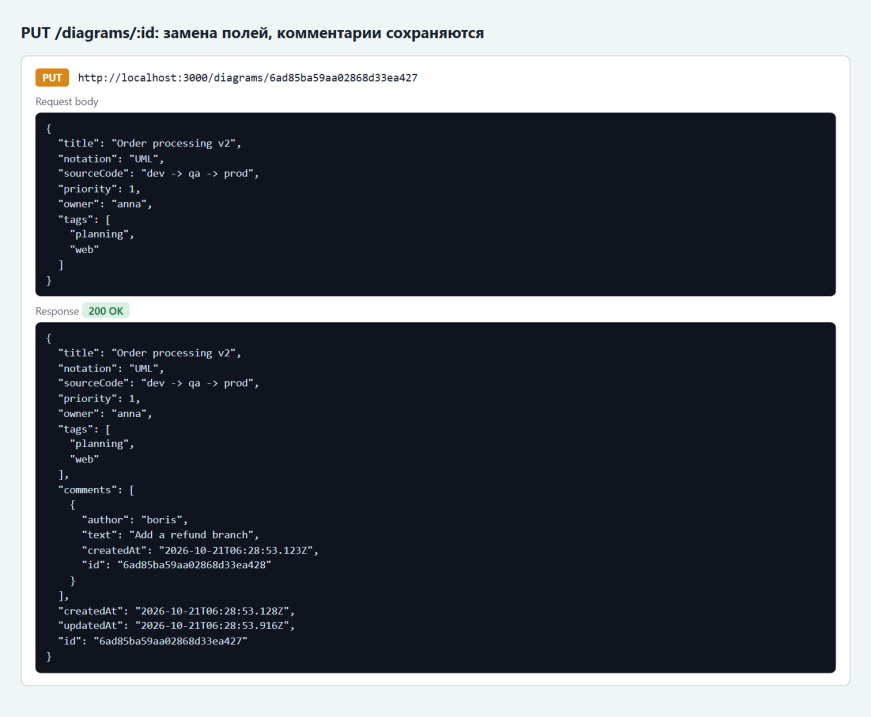



Рисунок 4 – Запрос PUT /diagrams/:id и ответ 200, записанные скриптом

Метод PATCH меняет одно поле и не трогает остальные, результат показан на рисунке 5.

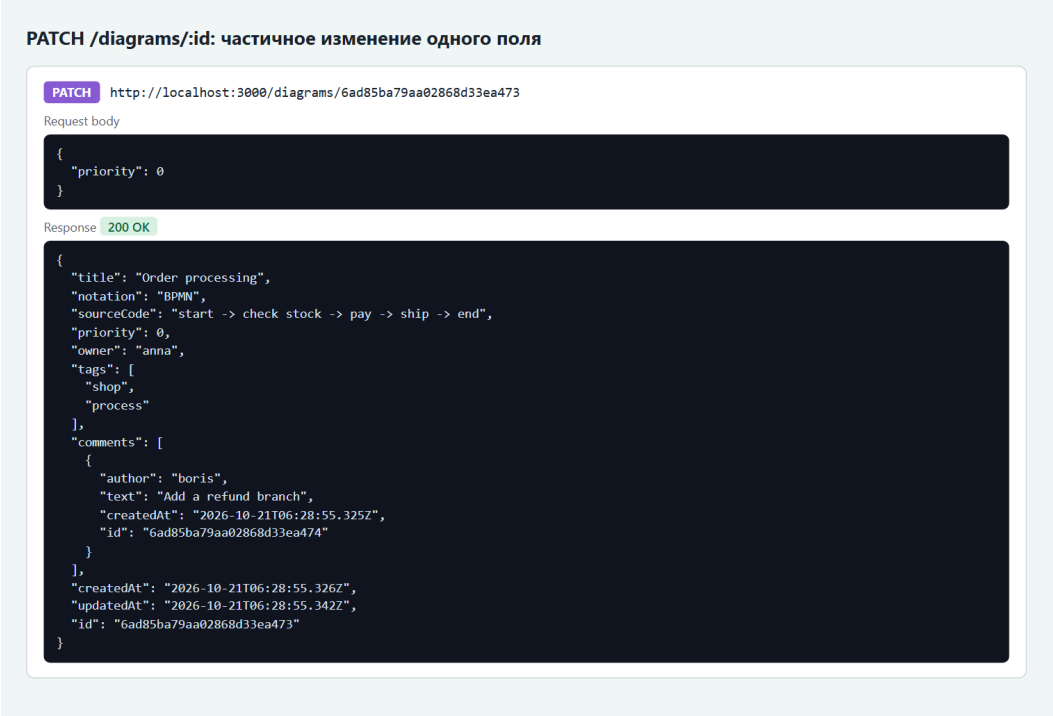



Рисунок 5 – Запрос PATCH /diagrams/:id с одним полем и ответ 200, записанные скриптом

Первое удаление возвращает 204, повторное удаление того же документа возвращает 404, что видно на рисунке 6.

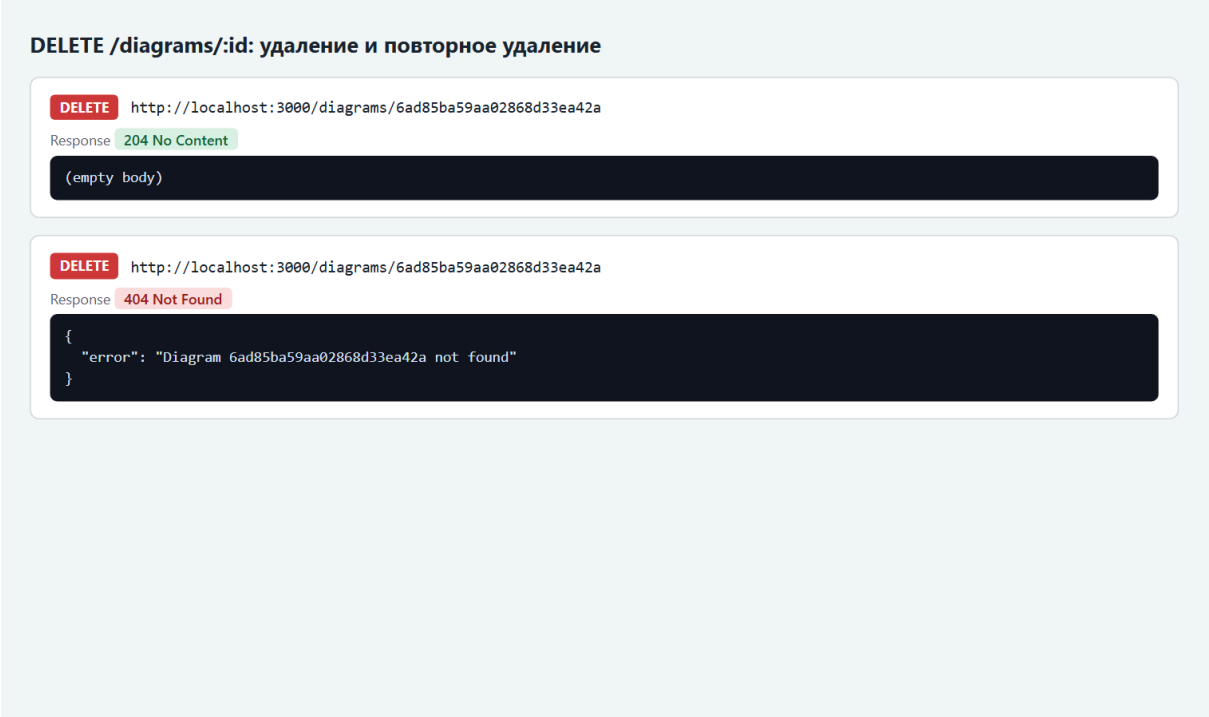



Рисунок 6 – Два запроса DELETE /diagrams/:id и ответы 204 и 404, записанные скриптом

Оператор `$push` добавляет комментарий в массив, как на рисунке 7.

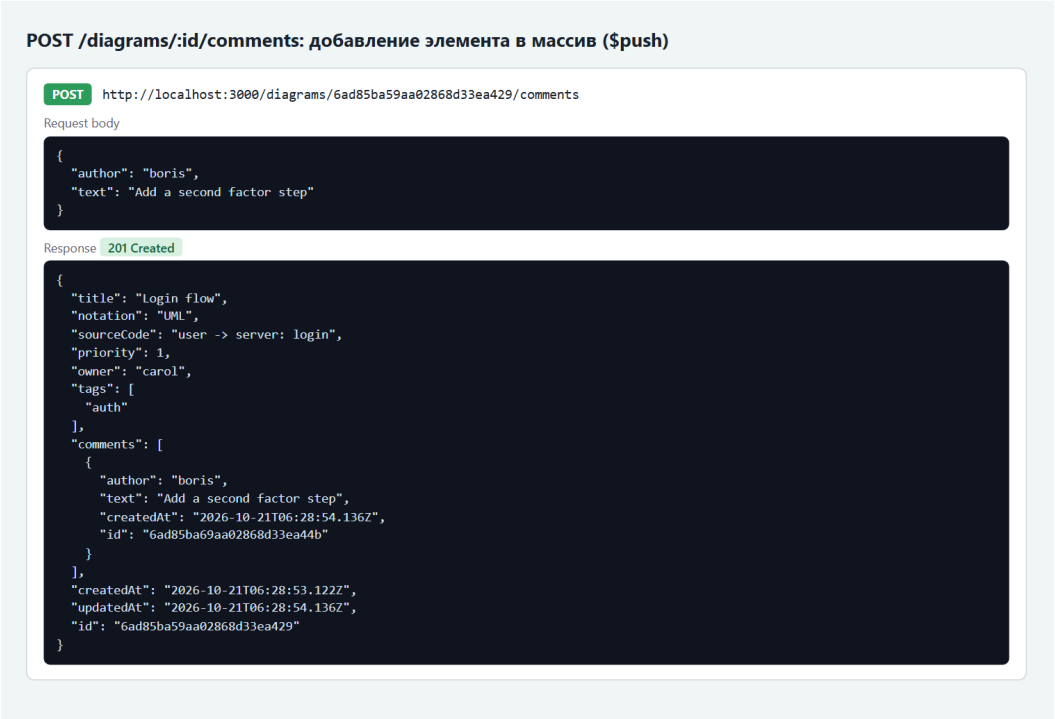



Рисунок 7 – Запрос POST /diagrams/:id/comments и ответ 201, записанные скриптом

Позиционный оператор `$` меняет текст одного элемента массива, как видно на рисунке 8.

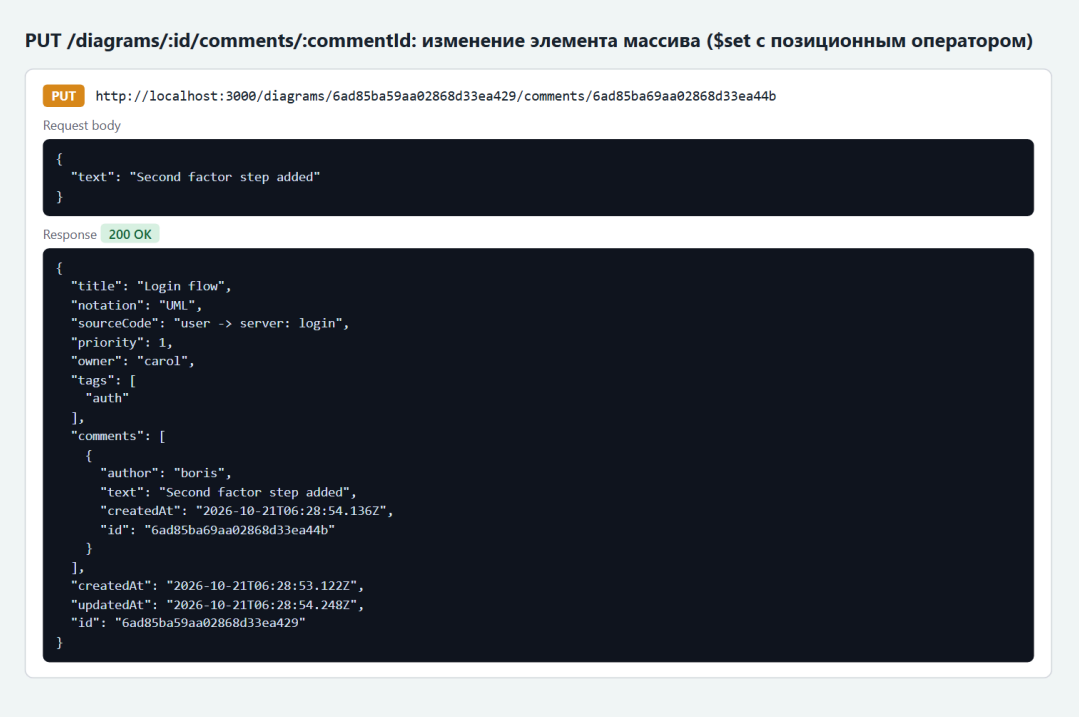



Рисунок 8 – Запрос PUT /diagrams/:id/comments/:commentId и ответ 200, записанные скриптом

Оператор `$pull` убирает элемент, и массив в ответе пуст, как на рисунке 9.

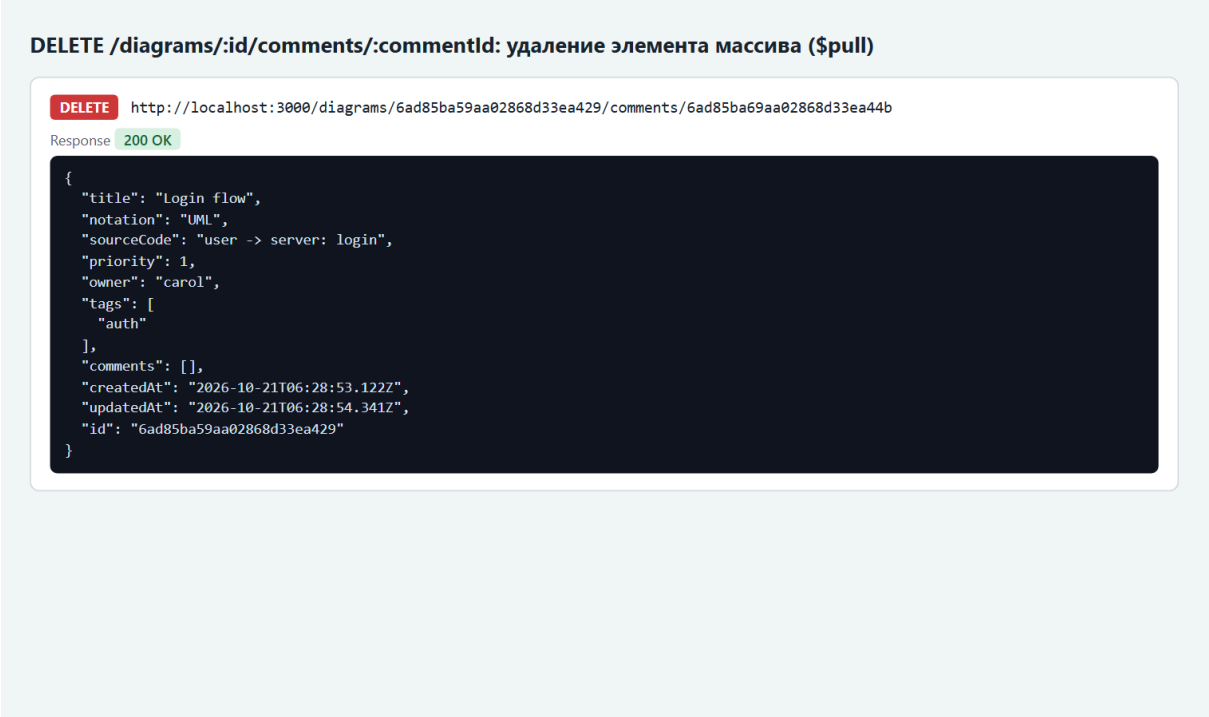



Рисунок 9 – Запрос DELETE /diagrams/:id/comments/:commentId и ответ 200, записанные скриптом

Фильтр по массиву тегов и владельцу возвращает только подходящие диаграммы, как на рисунке 10.

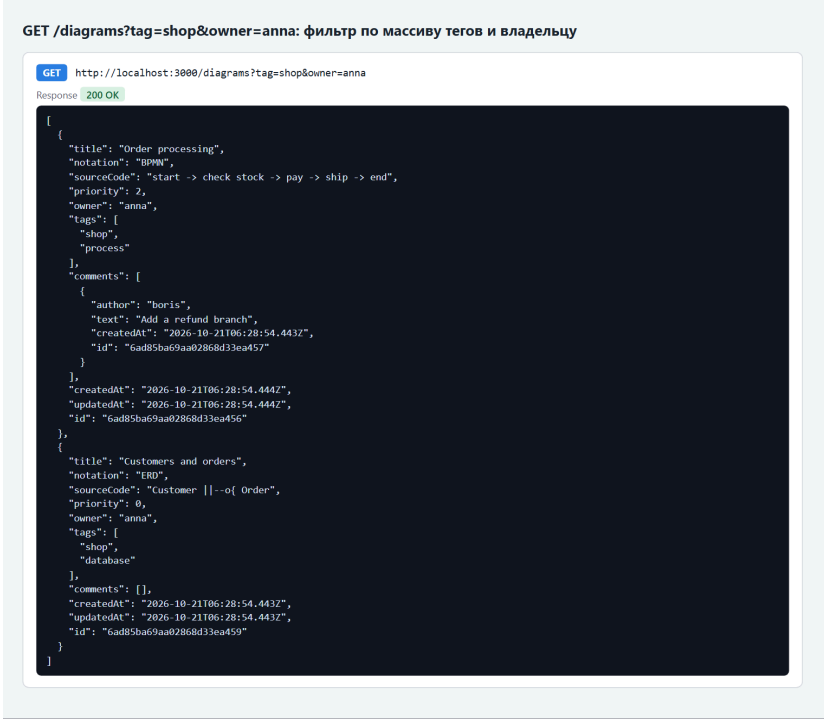



Рисунок 10 – Запрос GET /diagrams?tag=shop&owner=anna и ответ 200, записанные скриптом

Параметр `sort=title` упорядочивает список по названию, результат виден на рисунке 11.

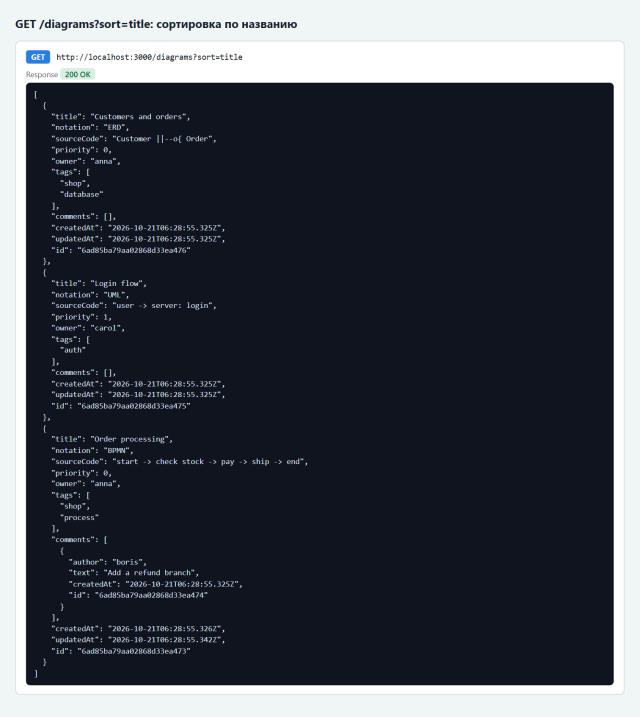



Рисунок 11 – Запрос GET /diagrams?sort=title и ответ 200, записанные скриптом

Параметр `q` ищет по текстовому индексу заголовка и исходного кода, как на рисунке 12.

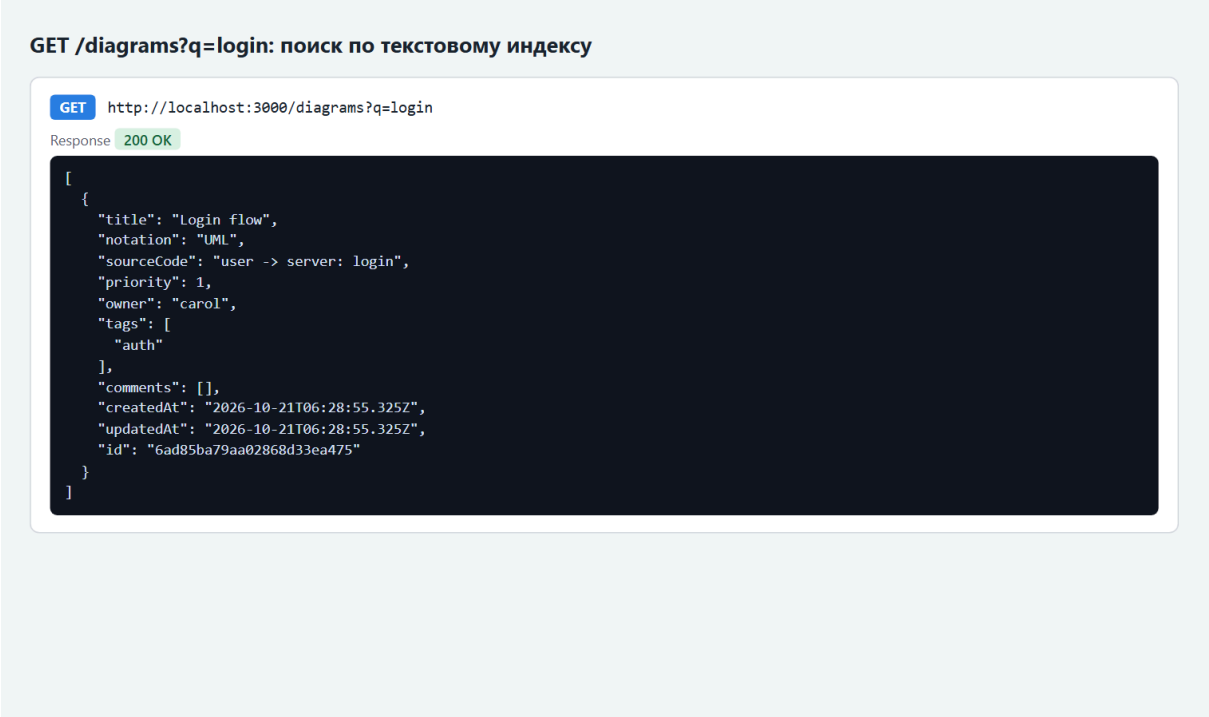



Рисунок 12 – Запрос GET /diagrams?q=login и ответ 200, записанные скриптом

Параметры `limit` и `skip` отдают одну страницу списка, что показано на рисунке 13.

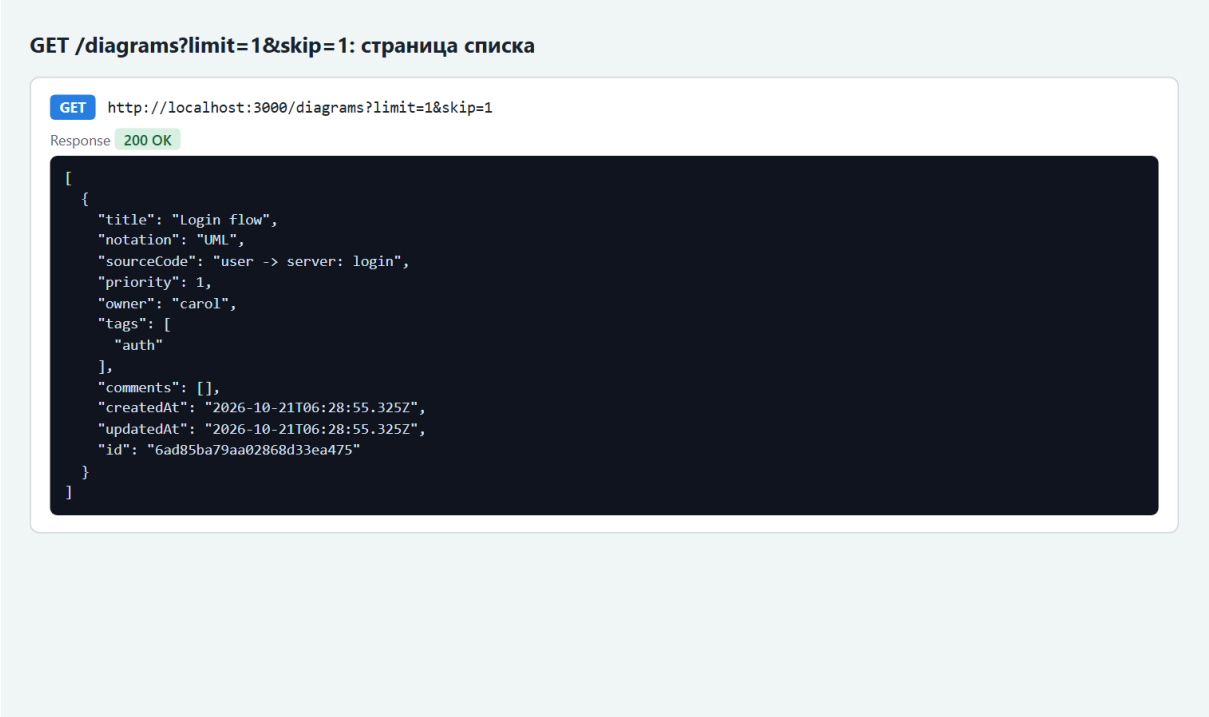



Рисунок 13 – Запрос GET /diagrams?limit=1&skip=1 и ответ 200, записанные скриптом

Маршрут `/diagrams/count` отдаёт число записей, а `/diagrams/stats` считает диаграммы по нотациям агрегацией, как видно на рисунке 14.

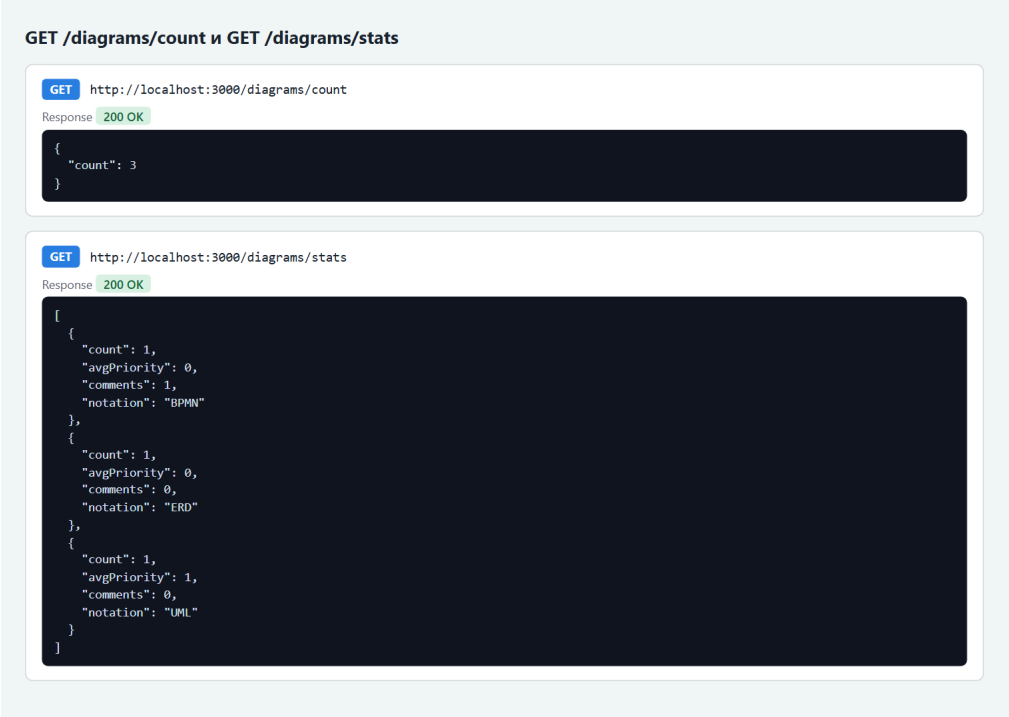



Рисунок 14 – Запросы GET /diagrams/count и GET /diagrams/stats с ответами 200, записанные скриптом

Сырой документ коллекции со вложенными тегами и комментариями показан на рисунке 15.

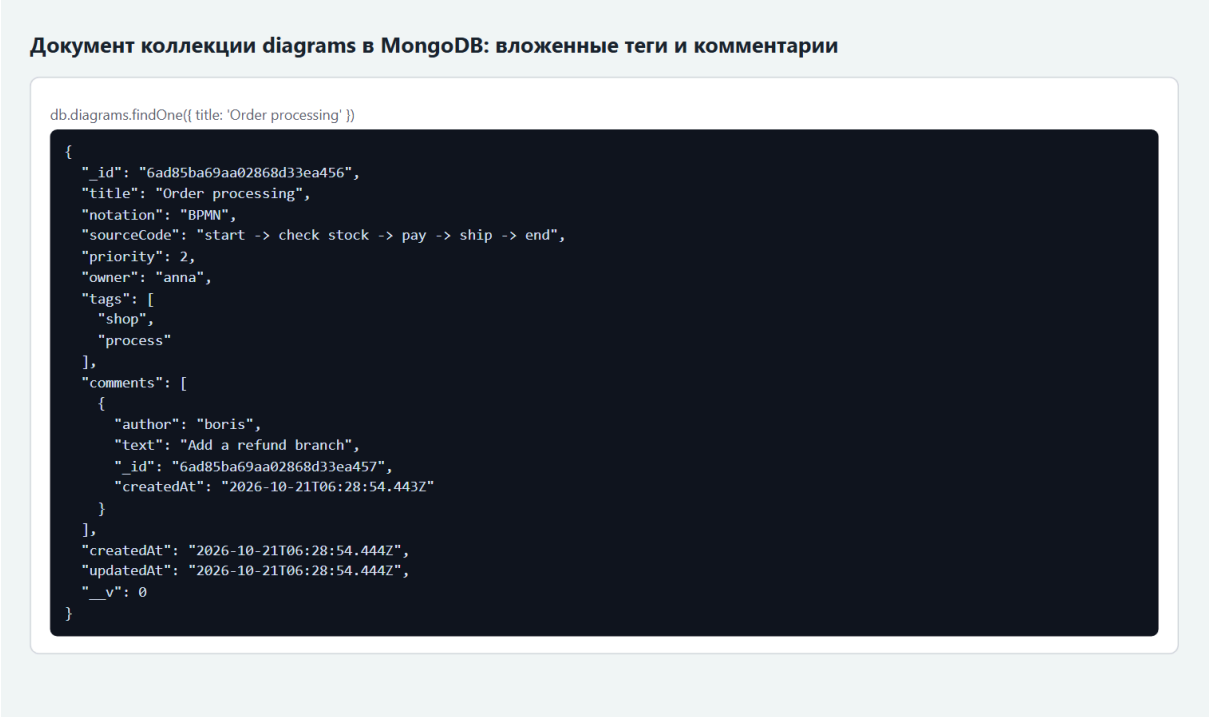



Рисунок 15 – Документ коллекции diagrams, прочитанный из MongoDB скриптом

Пустые и пробельные строки, неизвестная нотация и приоритет вне диапазона дают код 400 с перечнем полей, как на рисунке 16.

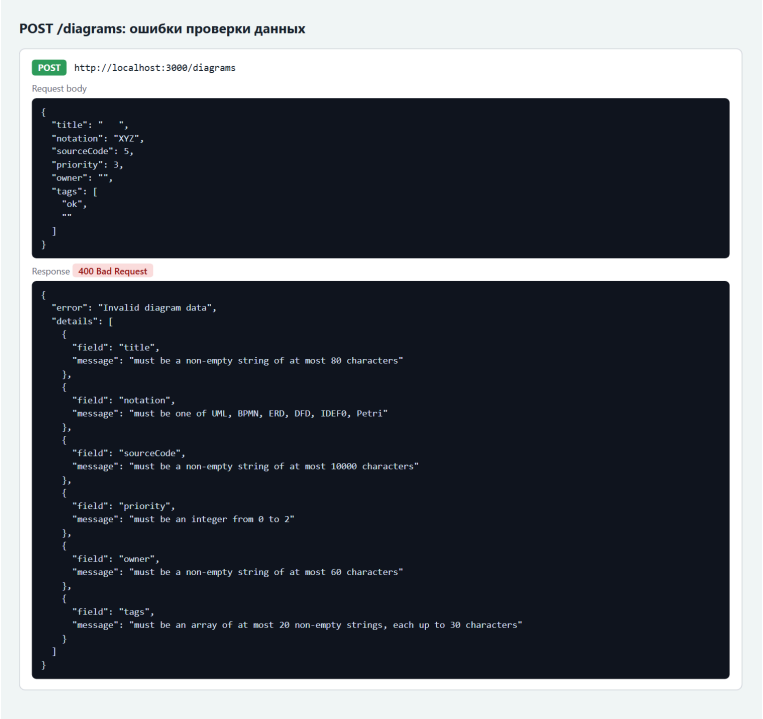
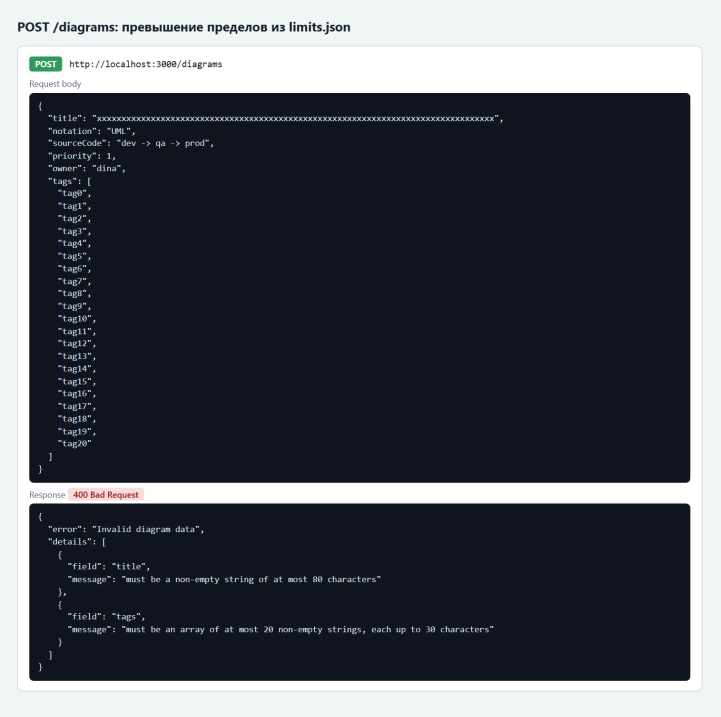



Рисунок 16 – Запрос POST /diagrams с неверными данными и ответ 400, записанные скриптом

Название длиннее 80 символов и 21 тег превышают пределы из `limits.json`, ответ показан на рисунке 17.


POST /diagrams: превышение пределов из limits.json
POST http://localhost:3000/diagrams
Request body
{
"title": "xxxx

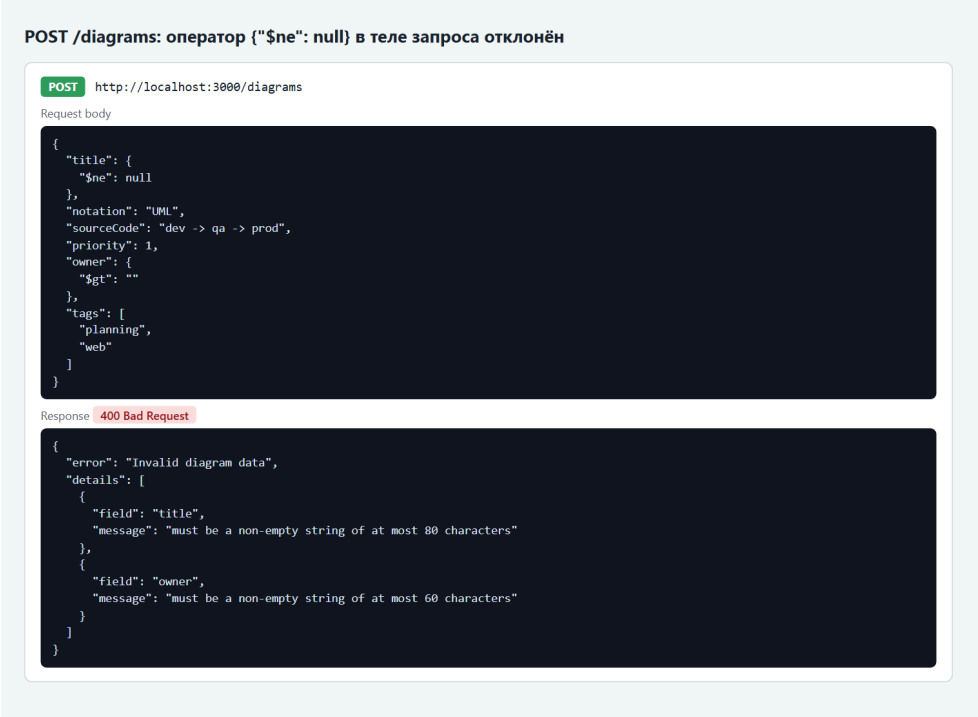



Рисунок 18 – Запрос POST /diagrams с операторами в теле и ответ 400, записанные скриптом

Параметр `owner[$ne]` и повторяющийся параметр `tag` отклоняются кодом 400, как на рисунке 19.

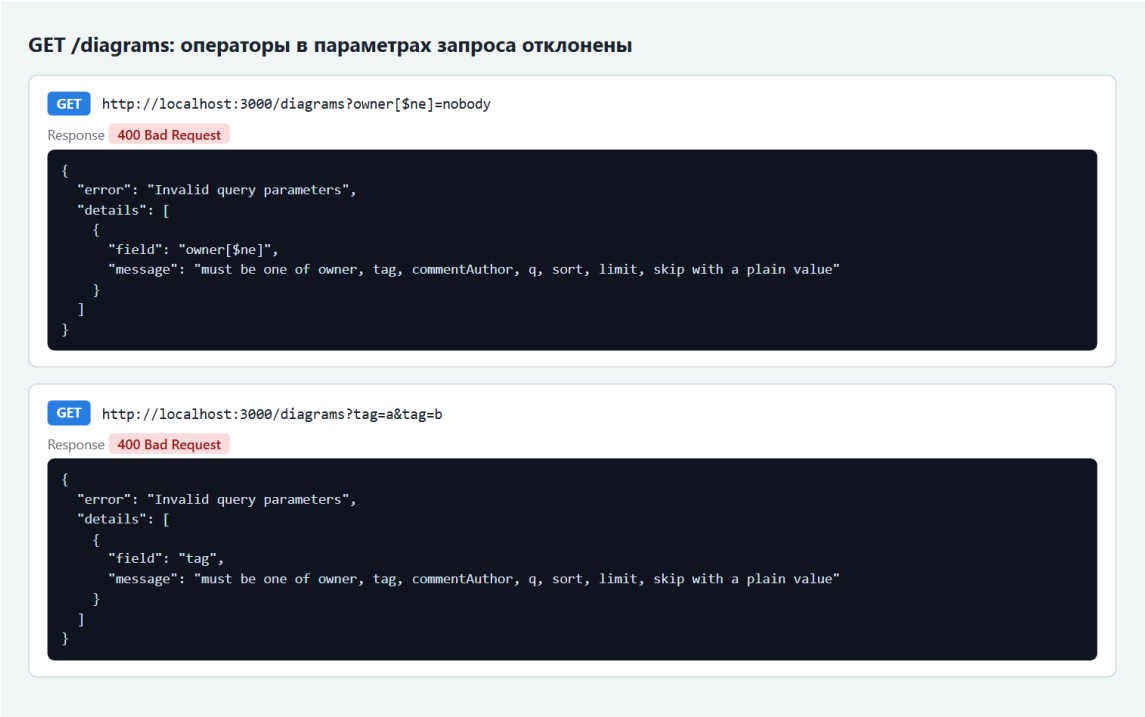



Рисунок 19 – Два запроса GET /diagrams с недопустимыми параметрами и ответы 400, записанные скриптом

Корректный, но неизвестный ObjectId даёт 404, а строка `abc` даёт 400, что видно на рисунке 20.

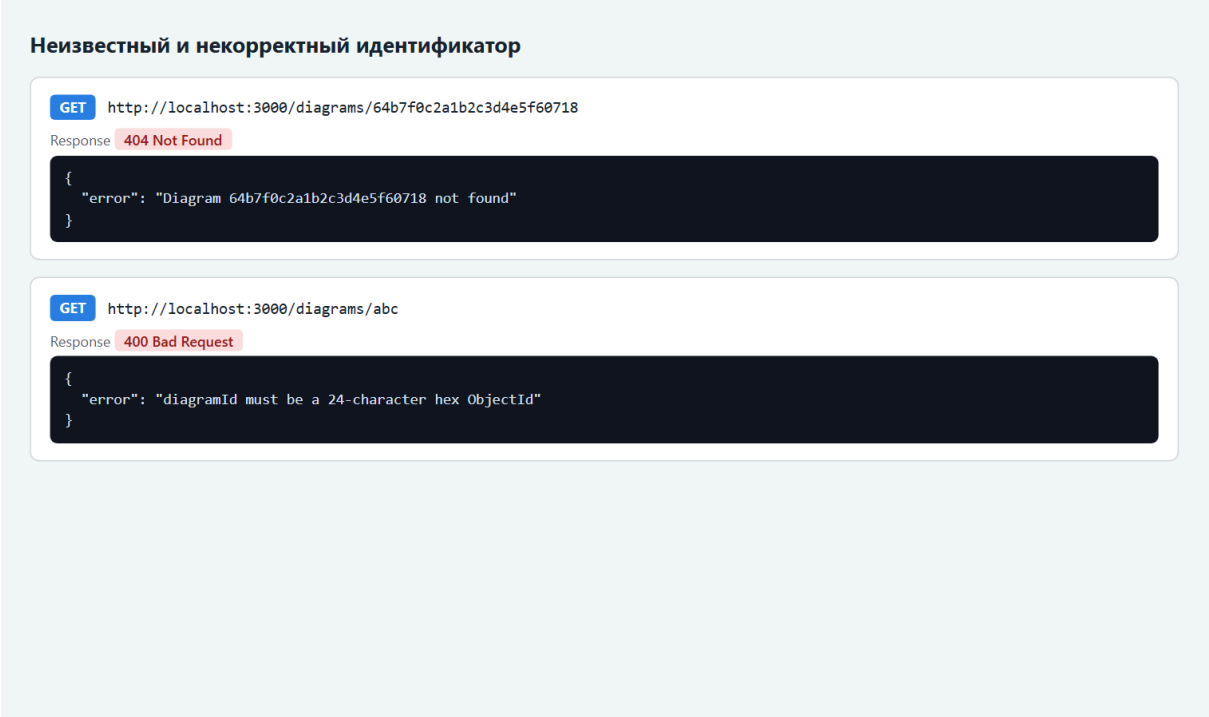



Рисунок 20 – Два запроса GET /diagrams/:id и ответы 404 и 400, записанные скриптом

Браузер Chrome открывает адрес `/diagrams?sort=title` работающего сервера и показывает ответ, как на рисунке 21.

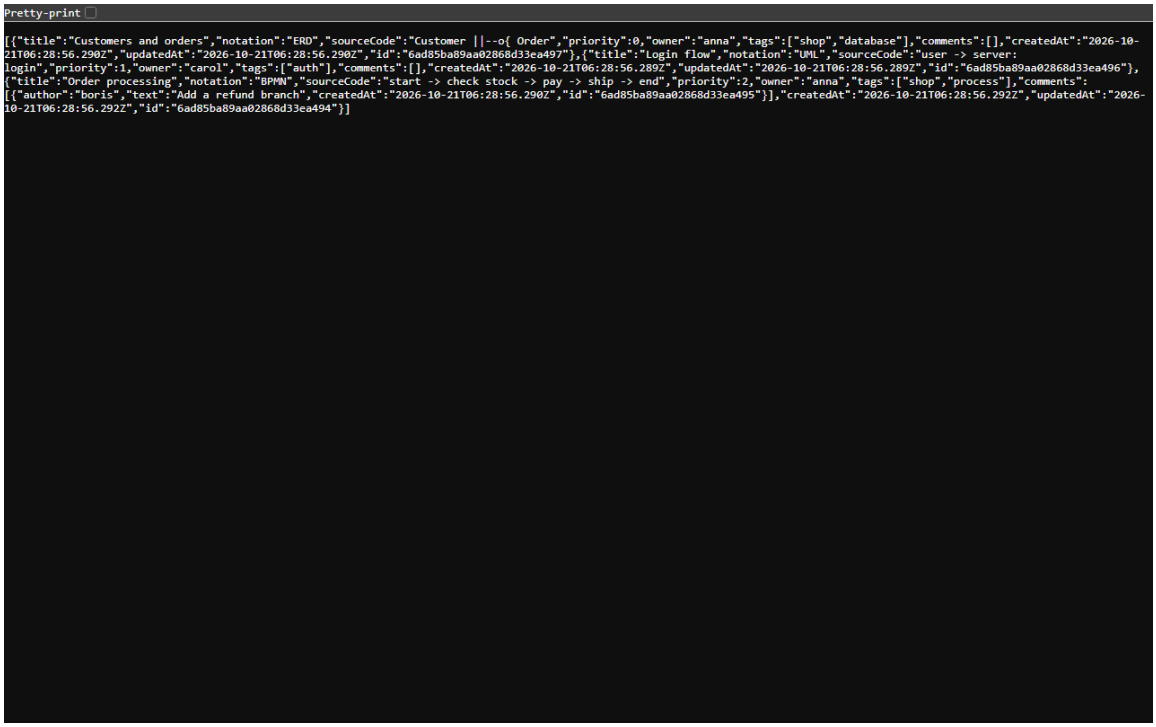



Рисунок 21 – Ответ GET /diagrams?sort=title в окне браузера Chrome

Неизвестный идентификатор в адресе браузера даёт ответ 404, как на рисунке 22.

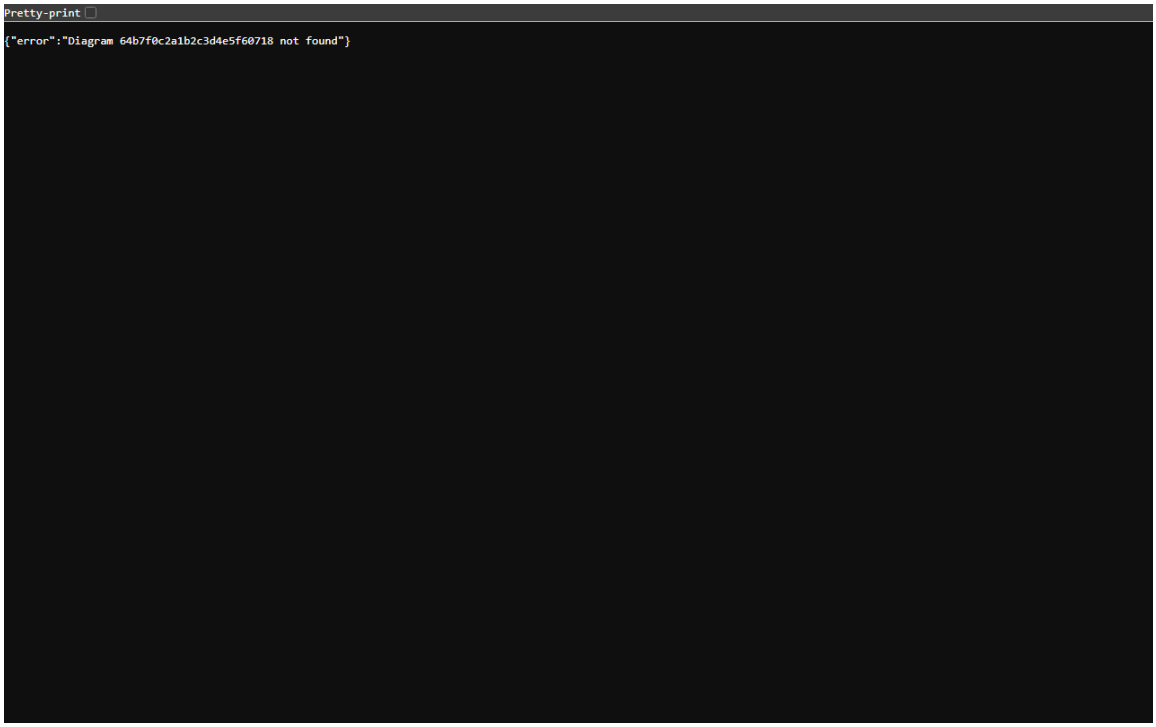



Рисунок 22 – Ответ GET /diagrams/64b7f0c2a1b2c3d4e5f60718 в окне браузера Chrome

Некорректный идентификатор `abc` даёт ответ 400, что видно на рисунке 23.

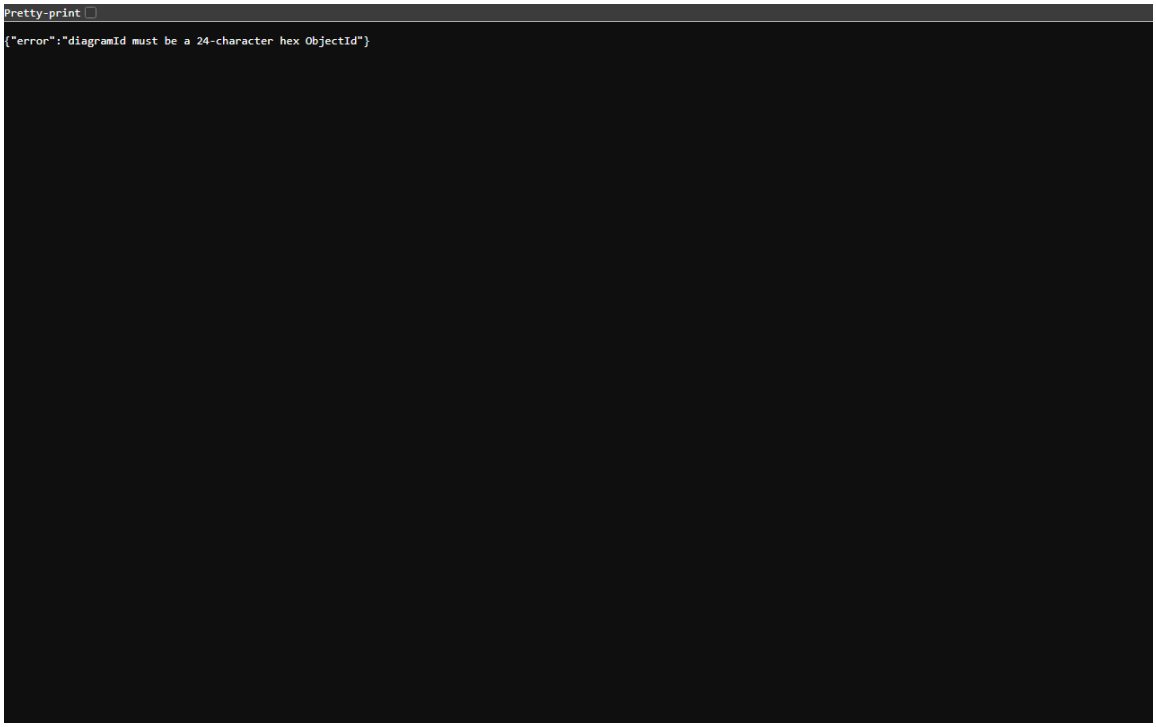



Рисунок 23 – Ответ GET /diagrams/abc в окне браузера Chrome

После остановки подключения к базе сервер не падает и отвечает JSON с кодом 503, как на рисунке 24.

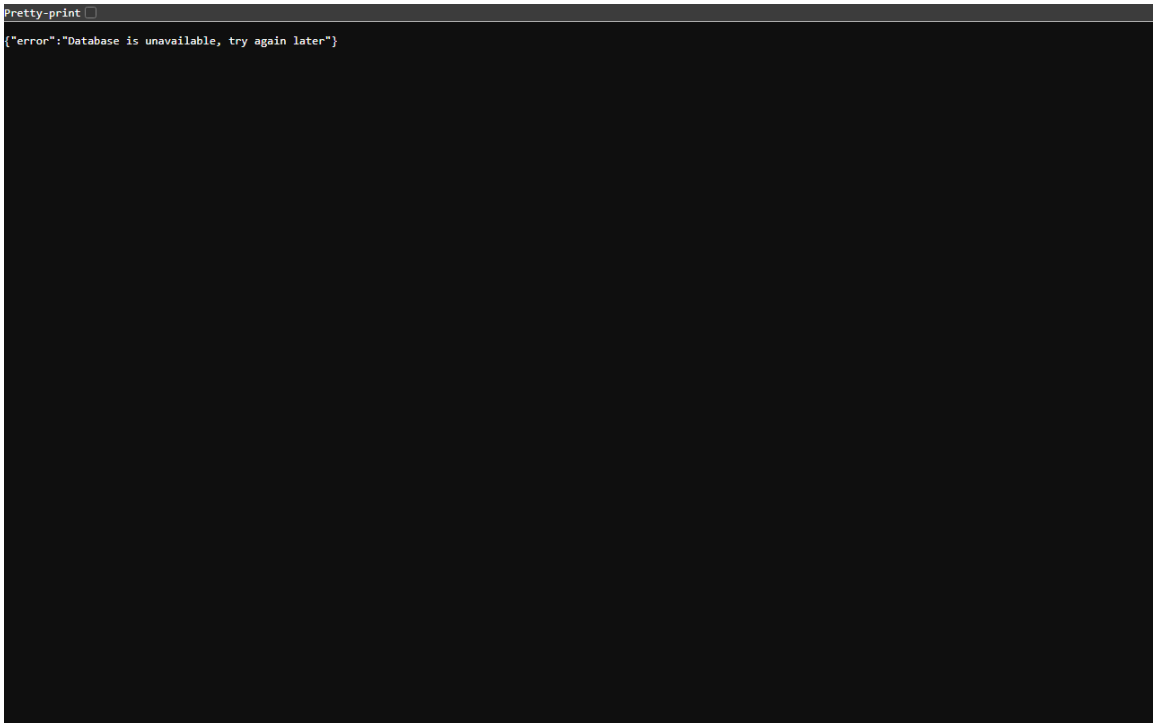



Рисунок 24 – Ответ GET /diagrams при отключённой MongoDB в окне браузера Chrome

Проверка `/health` при остановленной базе тоже возвращает 503, как на рисунке 25.

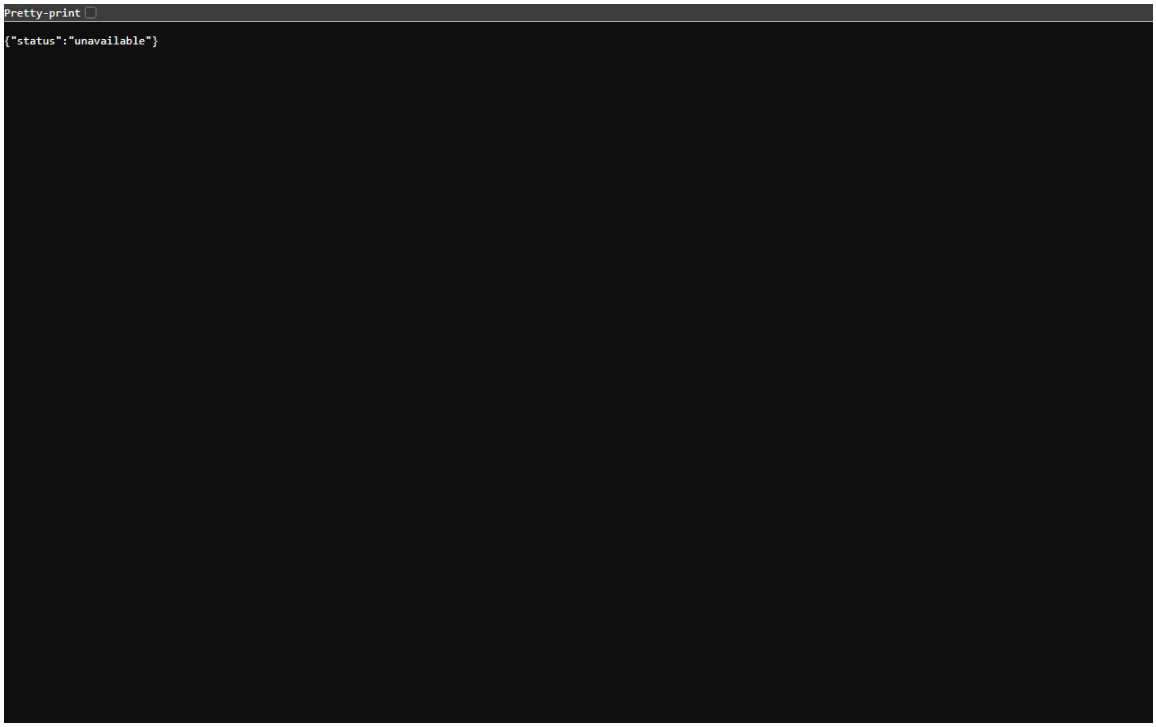



Рисунок 25 – Ответ GET /health при отключённой MongoDB в окне браузера Chrome

## ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

9. **Как создать индекс в Mongoose?** Параметр `index: true` у поля или `schema.index({ owner: 1 })`. Уникальность задаёт `unique: true`. В проекте индексы стоят на `owner` и `tags`.
10. **Что такое populate?** Метод, который заменяет ссылку `ObjectId` в документе данными из другой коллекции. В проекте он не нужен, потому что комментарии вложены, а не вынесены в отдельную коллекцию.
11. **Как искать, когда массив содержит значение?** Условие на поле-массив сравнивает значение с элементами: `Diagram.find({ tags: 'shop' })`, для нескольких значений есть `$in` и `$all`.

Листинг файла db.js приведён в приложении А. Листинг файла server.js приведён в приложении Б. Листинг файла diagram.js приведён в приложении В. Листинг файла diagrams.controller.js приведён в приложении Г. Листинг файла diagrams.routes.js приведён в приложении Д. Листинг файла seed.js приведён в приложении Е. Листинг файла limits.json приведён в приложении Ж.

# ЗАКЛЮЧЕНИЕ

В лабораторной работе сервер DSLKit из лабораторной работы 2 переведён с Sequelize на Mongoose. Коллекция `diagrams` хранится в MongoDB, маршруты CRUD сохранили адреса и формат ответов, а идентификаторы стали строками `ObjectId`. Теги и комментарии лежат внутри документа диаграммы, что заменяет несколько таблиц и соединений одним запросом. Для массива комментариев реализованы добавление, изменение элемента позиционным оператором и удаление. Строка подключения вынесена в переменную окружения, схема проверяет данные, а 144 теста подтверждают поведение API на временной базе. Не выполнено подключение к MongoDB Atlas и не приложены скриншоты.

**ПРИЛОЖЕНИЕ А**

(обязательное)

**Листинг файла db.js**

```
const mongoose = require('mongoose');

// The connection string comes from MONGODB_URI (see .env.exampl
e).
// MONGO_URI, the name used in the task text, is accepted as an
alias.
const connect = async (uri = process.env.MONGODB_URI || process.
env.MONGO_URI) => {
  if (!uri) throw new Error('MONGODB_URI is not set, copy .env.e
xample to .env');
  return mongoose.connect(uri, { serverSelectionTimeoutMS: 5000
});
};

module.exports = { connect, disconnect: () => mongoose.disconnec
t() };
```

## ПРИЛОЖЕНИЕ Б

(обязательное)

**Листинг файла server.js**

```
require('dotenv').config({ quiet: true });
const { createApp } = require('./src/app');
const models = require('./src/models');
const { connect } = require('./src/db');

const port = Number(process.env.PORT) || 3000;
const RETRY_MS = 5000;

// The server starts even when MongoDB is down: requests then ge
t a clear 503 answer
// while the connection is retried in the background, and the pr
ocess does not exit.
const connectWithRetry = async () => {
  try {
    await connect();
    console.log('MongoDB connected');
  } catch (error) {
    console.error(`Cannot connect to MongoDB (${error.message}),
retrying in ${RETRY_MS / 1000} s`);
    setTimeout(connectWithRetry, RETRY_MS).unref?.();
  }
};

createApp(models).listen(port, () => console.log(`Server running
on http://localhost:${port}`));
connectWithRetry();
```

## ПРИЛОЖЕНИЕ В
(обязательное)
**Листинг файла diagram.js**

```
const mongoose = require('mongoose');
const { NOTATIONS, PRIORITY, LIMITS } = require('./diagram.const
ants');

// Query filters are never trusted: operators sent as values are
stripped, unknown fields are ignored.
mongoose.set('sanitizeFilter', true);
mongoose.set('strictQuery', true);

// Responses expose "id" instead of Mongo's "_id" and "__v".
const toJSON = {
  virtuals: true,
  versionKey: false,
  transform: (_doc, ret) => {
    delete ret._id;
  },
};

// A comment lives inside the diagram document: no separate coll
ection, no join.
const commentSchema = new mongoose.Schema(
  {
    author: { type: String, required: true, trim: true, minlengt
h: 1, maxlength: LIMITS.author },
    text: { type: String, required: true, trim: true, minlength:
1, maxlength: LIMITS.commentText },
  },
  { timestamps: { createdAt: true, updatedAt: false }, toJSON, s
trict: true },
);

const diagramSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true, minlengt
h: 1, maxlength: LIMITS.title },
    notation: { type: String, required: true, enum: NOTATIONS },
    sourceCode: { type: String, required: true, trim: true, minl
ength: 1, maxlength: LIMITS.sourceCode },
    priority: {
      type: Number,
      required: true,
      min: PRIORITY.min,
      max: PRIORITY.max,
      default: PRIORITY.default,
      validate: { validator: Number.isInteger, message: 'priorit
y must be an integer' },
```

```
    },
    owner: { type: String, required: true, trim: true, minlengt
h: 1, maxlength: LIMITS.owner, index: true },
    tags: {
      type: [{ type: String, trim: true, minlength: 1, maxlengt
h: LIMITS.tag }],
      index: true,
      validate: { validator: (v) => v.length <= LIMITS.tags, mes
sage: 'too many tags' },
    },
    comments: { type: [commentSchema], validate: { validator:
(v) => v.length <= LIMITS.comments, message: 'too many comments'
} },
  },
  { timestamps: true, toJSON, strict: true },
);

// Multikey index on the nested field, used by ?commentAuthor=.
diagramSchema.index({ 'comments.author': 1 });

// Text index on the title and the source: used by ?q= through
$text.
diagramSchema.index({ title: 'text', sourceCode: 'text' });

const Diagram = mongoose.models.Diagram || mongoose.model('Diagr
am', diagramSchema);

module.exports = { Diagram };
```

## ПРИЛОЖЕНИЕ Г

(обязательное)

### Листинг файла diagrams.controller.js

```
const mongoose = require('mongoose');
const { badRequest, notFound } = require('../errors');
const { LIMITS } = require('../models/diagram.constants');

const missing = (id) => notFound(`Diagram ${id} not found`);
const missingComment = (id) => notFound(`Comment ${id} not found
`);

// ?owner=anna&tag=shop narrows the list; tag matches when the t
ags array contains the value.
// commentAuthor looks inside the nested comments array with the
dotted path comments.author.
const listFilter = ({ owner, tag, commentAuthor, q }) => ({
  ...(typeof q === 'string' && { $text: mongoose.trusted({ $sear
ch: q }) }),
  ...(typeof owner === 'string' && { owner }),
  ...(typeof tag === 'string' && { tags: tag }),
  ...(typeof commentAuthor === 'string' && { 'comments.author':
commentAuthor }),
});

const options = { new: true, runValidators: true };

// Express 5 forwards a rejected promise to the error handler, s
o no try/catch is needed here.
const createDiagramsController = ({ Diagram }) => ({
  list: async (req, res) => {
    const limit = Number(req.query.limit ?? LIMITS.listLimitMa
x);
    const skip = Number(req.query.skip ?? 0);
    const sortKey = req.query.sort;
    const sort = sortKey ? { [sortKey.replace('-', '')]: sortKe
y.startsWith('-') ? -1 : 1, _id: 1 } : { _id: 1 };
    const filter = listFilter(req.query);
    res.set('X-Total-Count', String(await Diagram.countDocuments
(filter)));
    res.json(await Diagram.find(filter).sort(sort).skip(skip).li
mit(limit));
  },

  // The record counter: how many diagrams match the same filter
s as the list.
  count: async (req, res) => res.json({ count: await Diagram.cou
ntDocuments(listFilter(req.query)) }),

  // Aggregation pipeline: group by notation, count the diagrams
```

```
and average the priority, then sort the groups.
  stats: async (_req, res) =>
    res.json(
      await Diagram.aggregate([
        { $group: { _id: '$notation', count: { $sum: 1 }, avgPri
ority: { $avg: '$priority' }, comments: { $sum: { $size: '$comme
nts' } } } },
        { $project: { _id: 0, notation: '$_id', count: 1, avgPri
ority: 1, comments: 1 } },
        { $sort: { notation: 1 } },
      ]),
    ),

  get: async (req, res, next) => {
    const diagram = await Diagram.findById(req.diagramId);
    return diagram ? res.json(diagram) : next(missing(req.diagra
mId));
  },

  create: async (req, res) => {
    const diagram = await Diagram.create(req.diagramFields);
    res.status(201).location(`/diagrams/${diagram.id}`).json(dia
gram);
  },

  replace: async (req, res, next) => {
    const diagram = await Diagram.findByIdAndUpdate(req.diagramI
d, req.diagramFields, options);
    return diagram ? res.json(diagram) : next(missing(req.diagra
mId));
  },

  // PATCH changes only the fields that were sent; comments stay
untouched.
  patch: async (req, res, next) => {
    const diagram = await Diagram.findByIdAndUpdate(req.diagramI
d, { $set: req.diagramFields }, options);
    return diagram ? res.json(diagram) : next(missing(req.diagra
mId));
  },

  remove: async (req, res, next) => {
    const diagram = await Diagram.findByIdAndDelete(req.diagramI
d);
    return diagram ? res.status(204).end() : next(missing(req.di
agramId));
  },

  // $push appends to the nested array in one atomic update. The
push runs only while the array has room:
  // update validators do not check the array length on $push, s
o the limit is part of the filter.
```

```
  addComment: async (req, res, next) => {
    // sanitizeFilter is on, so the operator is marked trusted:
it is the server's own, not user input.
    const room = { [`comments.${LIMITS.comments - 1}`]: mongoos
e.trusted({ $exists: false }) };
    const diagram = await Diagram.findOneAndUpdate({ _id: req.di
agramId, ...room }, { $push: { comments: req.commentFields } },
options);
    if (diagram) return res.status(201).json(diagram);
    const exists = await Diagram.exists({ _id: req.diagramId });
    return exists ? next(badRequest(`A diagram holds at most ${L
IMITS.comments} comments`)) : next(missing(req.diagramId));
  },

  // The positional operator $ updates only the array element ma
tched by the filter.
  updateComment: async (req, res, next) => {
    const diagram = await Diagram.findOneAndUpdate(
      { _id: req.diagramId, 'comments._id': req.commentId },
      { $set: { 'comments.$.text': req.commentFields.text } },
      options,
    );
    return diagram ? res.json(diagram) : next(missingComment(re
q.commentId));
  },

  removeComment: async (req, res, next) => {
    const diagram = await Diagram.findOneAndUpdate(
      { _id: req.diagramId, 'comments._id': req.commentId },
      { $pull: { comments: { _id: req.commentId } } },
      options,
    );
    return diagram ? res.json(diagram) : next(missingComment(re
q.commentId));
  },
});

module.exports = { createDiagramsController };
```

## ПРИЛОЖЕНИЕ Д
(обязательное)
### Листинг файла diagrams.routes.js

```
const { Router } = require('express');
const { HttpError } = require('../errors');
const { validatePatch, validateListQuery, parseId, parseCommentI
d, validateDiagram, validateComment, validateCommentText } = req
uire('../middleware/validate');

const methodNotAllowed = (allow) => (_req, res, next) => {
  res.set('Allow', allow);
  next(new HttpError(405, `Method not allowed, use ${allow}`));
};

const createDiagramsRouter = (controller) => {
  const router = Router();
  router.route('/').get(validateListQuery, controller.list).post
(validateDiagram, controller.create).all(methodNotAllowed('GET,
POST'));
  router.route('/count').get(validateListQuery, controller.coun
t).all(methodNotAllowed('GET'));
  router.route('/stats').get(controller.stats).all(methodNotAllo
wed('GET'));
  router
    .route('/:id')
    .get(parseId, controller.get)
    .put(parseId, validateDiagram, controller.replace)
    .patch(parseId, validatePatch, controller.patch)
    .delete(parseId, controller.remove)
    .all(methodNotAllowed('GET, PUT, PATCH, DELETE'));
  router.route('/:id/comments').post(parseId, validateComment, c
ontroller.addComment).all(methodNotAllowed('POST'));
  router
    .route('/:id/comments/:commentId')
    .put(parseId, parseCommentId, validateCommentText, controlle
r.updateComment)
    .delete(parseId, parseCommentId, controller.removeComment)
    .all(methodNotAllowed('PUT, DELETE'));
  return router;
};

module.exports = { createDiagramsRouter };
```

## ПРИЛОЖЕНИЕ Е
(обязательное)
## Листинг файла seed.js

```
require('dotenv').config({ quiet: true });
const { Diagram } = require('../src/models');
const { connect, disconnect } = require('../src/db');

const { ROWS: demo } = require('./seed-diagrams');

// Replaces the demo documents by title, so the script can be ru
n many times.
const seed = async () => {
  await connect();
  await Diagram.deleteMany({ title: { $in: demo.map((d) => d.tit
le) } });
  const created = await Diagram.create(demo);
  console.log(`Seeded ${created.length} diagrams`);
  await disconnect();
};

seed().catch((error) => {
  console.error('Seed failed:', error.message);
  process.exit(1);
});
```

## ПРИЛОЖЕНИЕ Ж

(обязательное)

**Листинг файла limits.json**

```
{
  "title": 80,
  "owner": 60,
  "sourceCode": 10000,
  "tags": 20,
  "tag": 30,
  "comments": 100,
  "author": 60,
  "commentText": 1000,
  "priorityMin": 0,
  "priorityMax": 2,
  "priorityDefault": 1,
  "listLimitMax": 100,
  "listSkipMax": 100000,
  "pathLog": 200,
  "logRetentionDays": 30
}
```